

Diffusion LLMs Can Do Faster-Than-AR Inference via Discrete Diffusion Forcing

2025.08.31

딥러닝논문읽기 자연어처리팀

딥러닝논문읽기

임다예슬

Overview

D2F = 기존 LLM (AR)의 KV 캐시를 활용 + Diffusion의 병렬성 → 정확성, 속도 모두 확보하여 dLLM을 실용적인 대안으로 제안

● 문제의식

- AR* LLMs: 품질 높지만 토큰을 순차적으로 생성 → 느림

* AR: Autoregressive

- dLLM*: 병렬 예측 가능하여 잠재적 속도 개선 가능 but KV 캐시 미활용 → 실제 속도는 AR보다 느림

* dLLM: Diffusion LLM

● 핵심 아이디어(D2F*)

* D2F: Discrete Diffusion Forcing

- Block-wise Autoregression : 블록 間 순차적 디코딩 → KV 캐시 사용 가능하여 속도 개선
- Inter-block Parallel Decoding: 블록 內 병렬 디코딩 → dLLM의 장점을 취함
- Asymmetric Distillation (KL Loss) → Teacher(전체 문맥) vs Student(부분 문맥)

● 성과

- 세계 최초 AR LLM보다 빠른 오픈소스 dLLM (GSM8K: LLaMA3보다 2.5X 빠름)
- 기존 dLLM 대비 최대 50X 속도 개선 (성능 유지)

D2F-LLaDA-Instruct-8B  **LLaMA3-Instruct-8B**

Motivation

- dLLM은 병렬 토큰 생성이 가능하여 잠재적 속도 개선이 가능한 모델
 - 그러나, 순차적 attention이 일어나지 않아 KV 캐시 미활용으로 인해 실제 속도는 AR보다 느림
- * Diffusion LLMs (dLLM): 모든 토큰을 한번에 병렬적으로 예측하기 위해 마스크된 문장을 점점 복원해가는 언어 모델

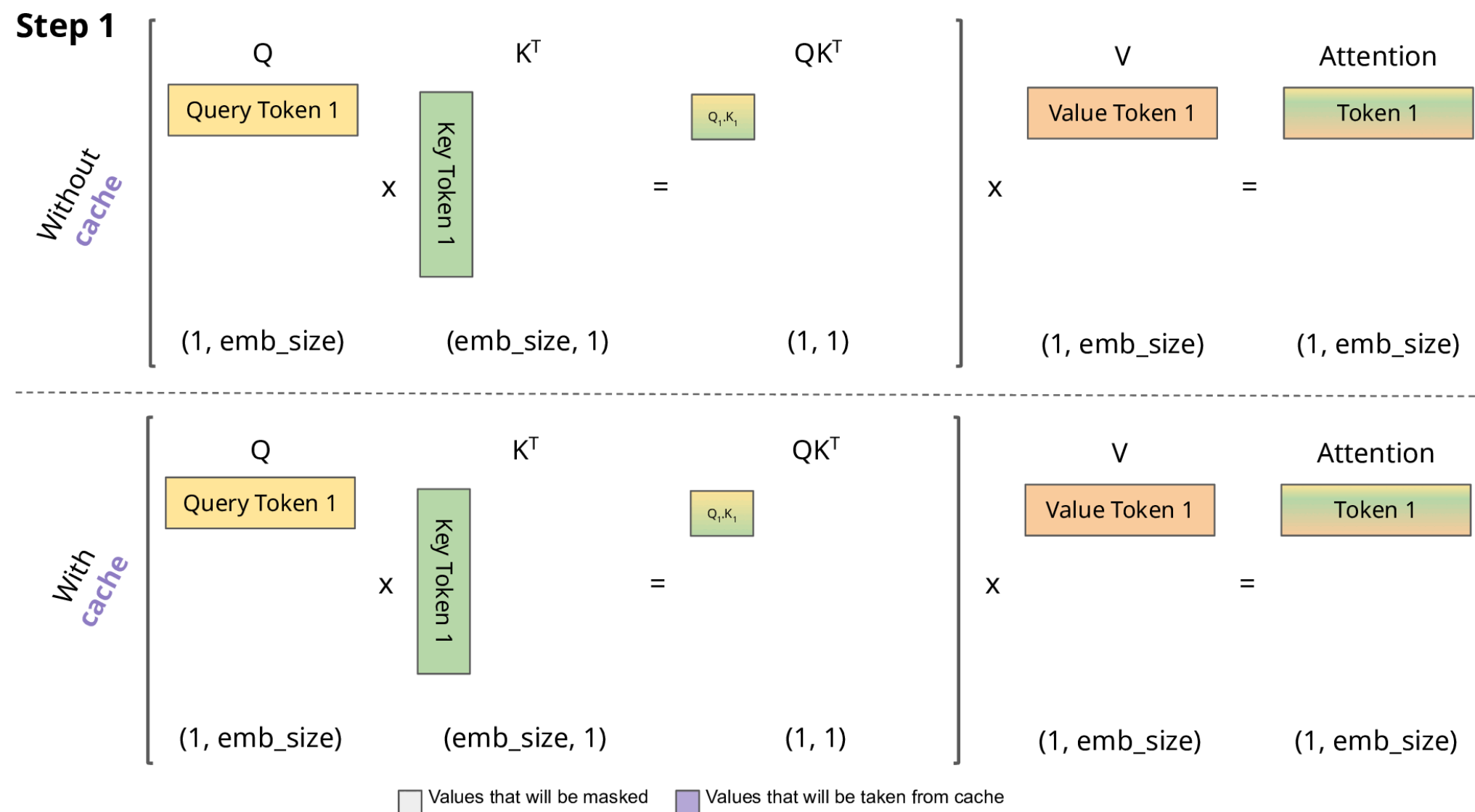
Prompt: *Explain what artificial intelligence is.*

,

Purpose

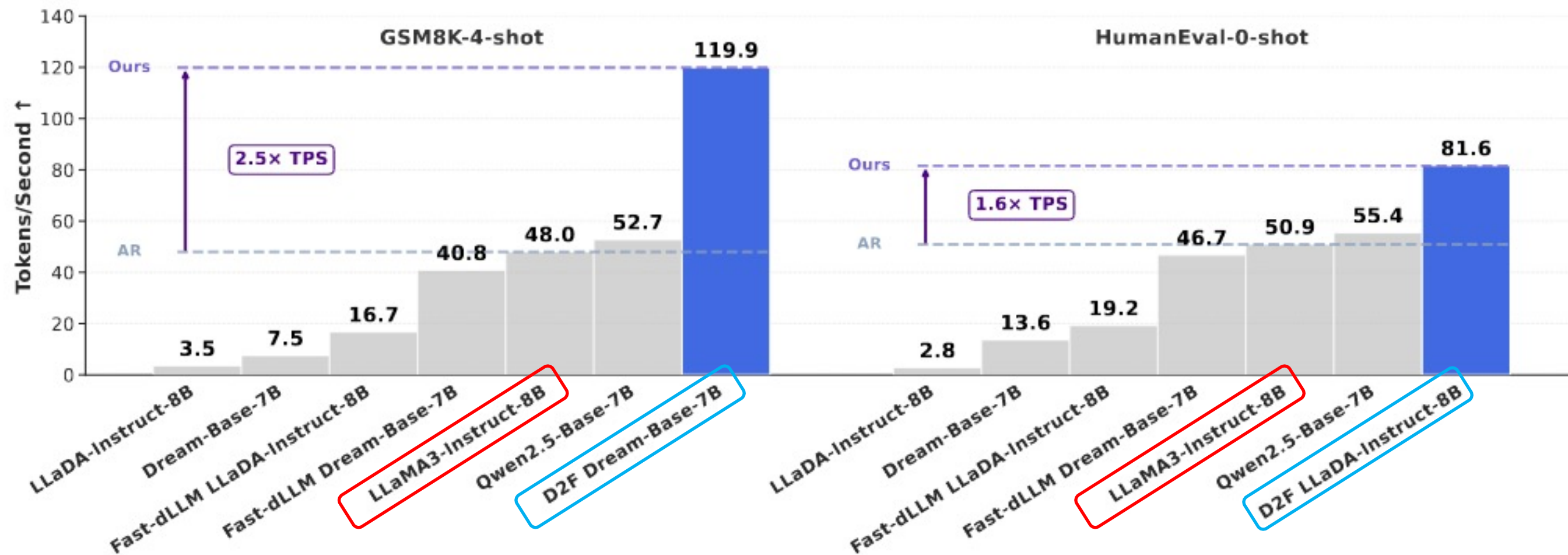
- dLLM을 AR보다 빠른 추론 모델로 가속화하자!
- KV 캐시와 병렬성 모두 활용할 수 있는 새로운 패러다임 제안

* KV cache: 디코딩 과정에서 이전 단계에서 이미 계산된 Key/Value 행렬을 저장(caching)해 두고, 새로 들어온 토큰의 Query만 계산하여 효율적으로 attention을 수행하는 최적화 기법.



Key Contributions

- D2F (Discrete Diffusion Forcing) 제안
- 최초로 AR LLM (LLAMA3) 보다 빠른 오픈소스 dLLM 달성
- 파이프라인 병렬 디코딩 + Asymmetric Distillation 기법 제시
- 성능 유지하면서 기존 dLLM 대비 최대 50배 가속 달성



Background

Autoregressive LLM vs diffusion LLM

AR LLMs

- 순차적 토큰 디코딩 → 느림
- KV 캐시 사용 가능 → 효율적

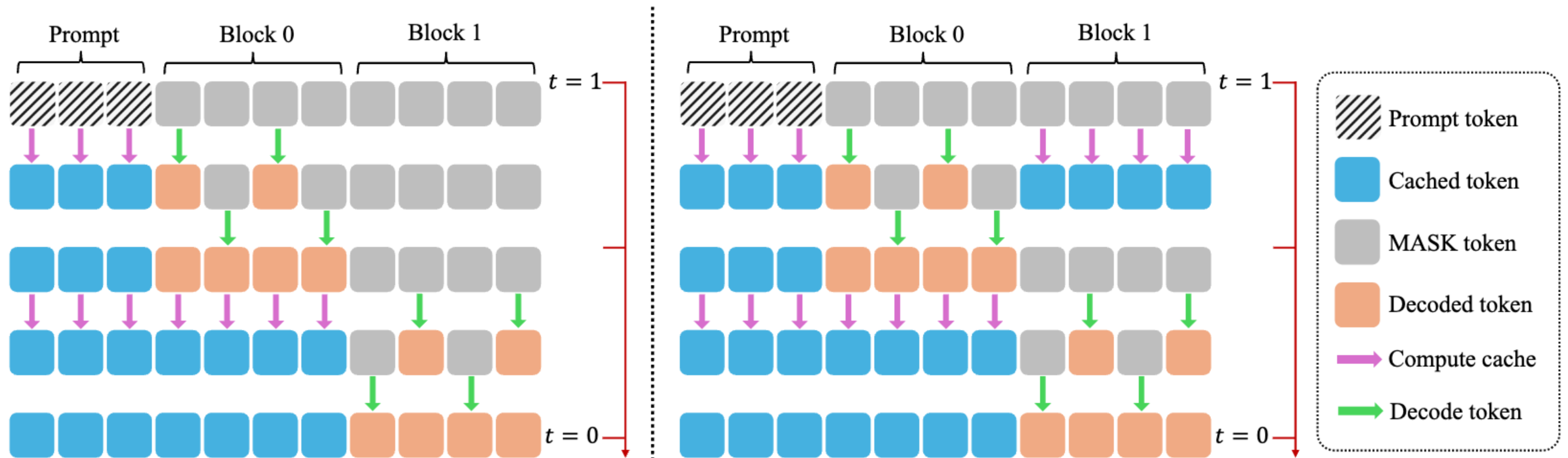
dLLMs

- 완전히 마스킹된 시퀀스를 점진적으로 복원(denoising)
- 병렬 생성 가능 (잠재적으로는 속도 경쟁력 있음)
- 양방향 어텐션 사용 → 문맥 이해력 ↑
- 단점: KV 캐시 사용 불가 → 속도 느림

Previous work - Acceleration of dLLMs

- Caching based method, Sampling optimization

- 예: Fast-dLLM



(a) Prefix KV Cache for block-wise generation.

(b) **DualCache**: Bidirectional KV cache contains prefix and suffix Cache.

- 본 논문과 같이 KV 캐시 활용을 위해 블록 단위 디코딩 진행
- Confidence level이 높은 토큰만 미리 디코딩하는 방식으로 효율화
- 한계: AR LLM의 속도를 뛰어넘지 못함

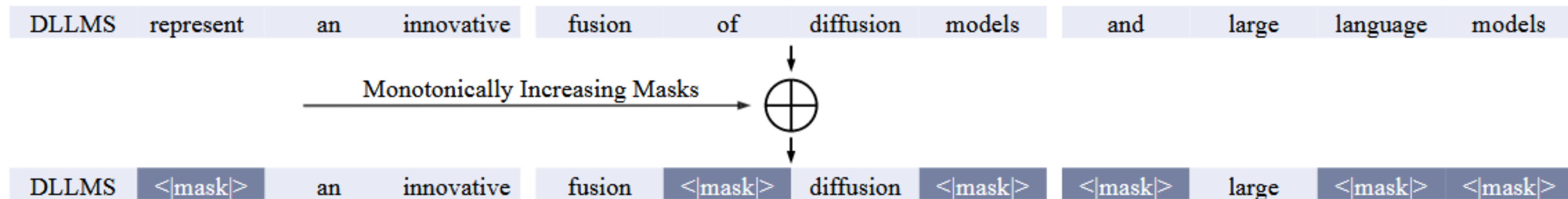
Method (1) Discrete Diffusion Forcing (D2F)

(1) Block-level AR generation : KV cache 사용으로 컴퓨팅 리소스 절감

(2) Inter-block parallel decoding: 모델이 일부만 복원된 앞 블록을 기반으로 뒷 블록을 예측하도록 훈련

→ 한 번의 단계에서 더 많은 토큰 디코딩 가능

두 번째 기능 구현을 위하여 D2F 사용, 일부 복원된 prefix에 조건부로 현재 블록을 denoising하도록 모델을 훈련함.



- 시퀀스 Y^0 를 크기 k 인 N 개 블록으로 분할
- 순방향 과정: 노이즈 $t_1 < t_2 < \dots < t_N$ 적용
- 역방향 과정 : 모델 p_θ 는 다음 수식으로 학습됨

$$p_\theta(Y^0|Y^t) = \prod_{i=1}^N p_\theta(Y_{B_i}^0 | Y_{B_1}^{t_1}, \dots, Y_{B_i}^{t_i})$$

블록 단위로 순차적 예측

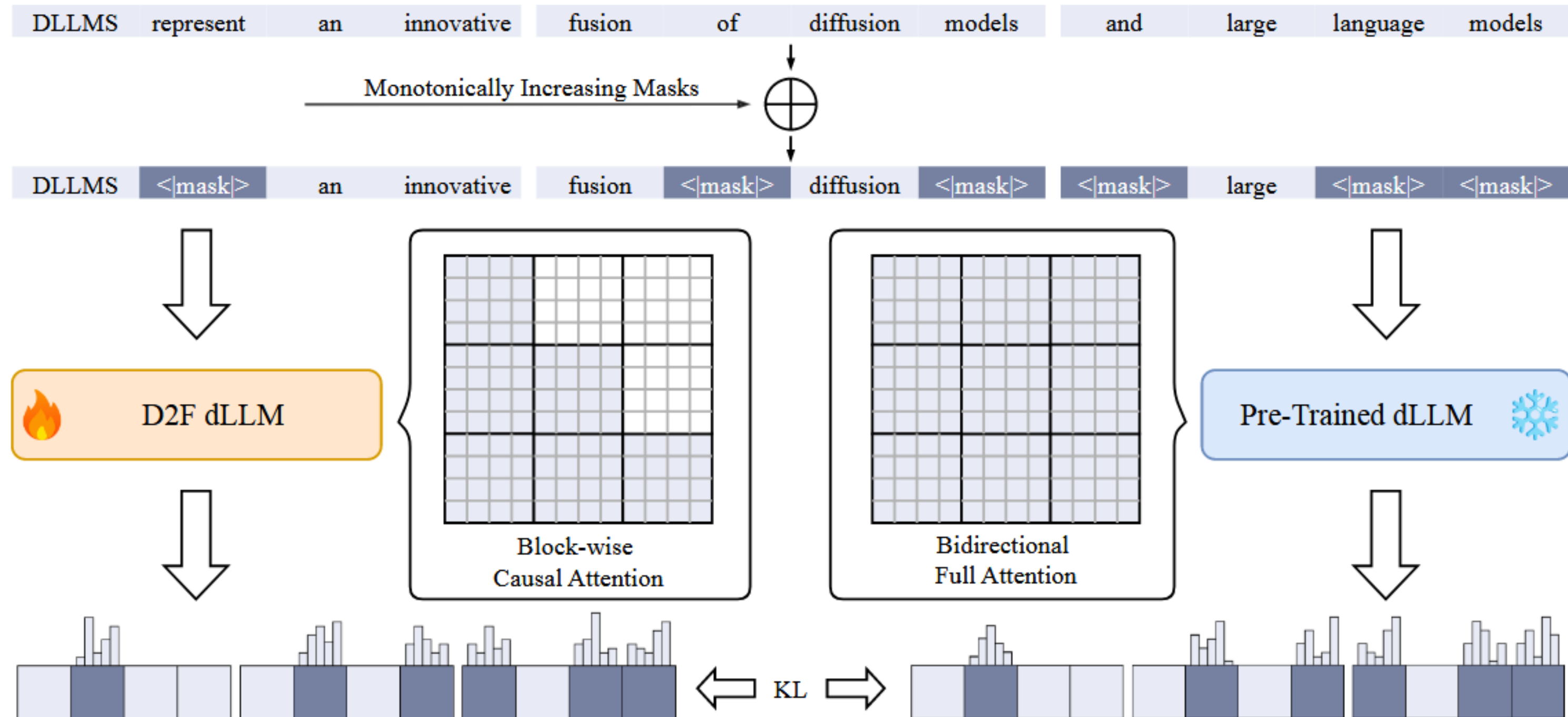
노이즈가 낀 입력 Y^t 로부터 원래 시퀀스 Y^0 를 복원할 확률

부분적으로 denoise된 앞 블록

→ 앞 블록 완성과 동시에 뒷 블록 진행 가능 (블록 間 병렬 디코딩 가능)

Method (2) Asymmetric Distillation

- Pre-trained dLLM으로부터 distillation(지식 증류)을 통해 D2F dLLM을 획득



Method (2) Asymmetric Distillation

- Pre-trained dLLM으로부터 distillation(지식 증류)을 통해 D2F dLLM을 획득
- Teacher(p_{ϕ^-}): 기존의 양방향 어텐션 dLLM
- Student(p_{θ}): D2F dLLM, Teacher 모델의 가중치로 초기화
- 손실 함수

$$\mathcal{L}_{\text{D2F}} = \mathbb{E}_{t_1 < \dots < t_N} \left[\sum_{i=1}^N D_{\text{KL}} \left(p_{\theta}(Y_{B_i}^0 | Y_{B_1}^{t_1}, \dots, Y_{B_i}^{t_i}) \parallel p_{\phi^-}(Y_{B_i}^0 | Y_{B_1}^{t_1}, \dots, Y_{B_N}^{t_N}) \right) \right]$$

Student 분포 :
1 번째 블록 앞부분까지만 부분복원 조건Teacher 분포 :
모든 noisy 블록을 다 본 상태에서 예측

- KL Divergence : Student가 Teacher 분포를 모방
- Teacher은 전체 문맥, Student는 제한된 문맥을 본다는 관점에서 asymmetric distillation
- Student 모델은 앞 블록이 완전히 복원되지 않더라도 Teacher처럼 뒷 블록을 잘 예측하는 능력을 학습하게 됨

Method (2) Asymmetric Distillation

Algorithm 1 Asymmetric Distillation for D2F

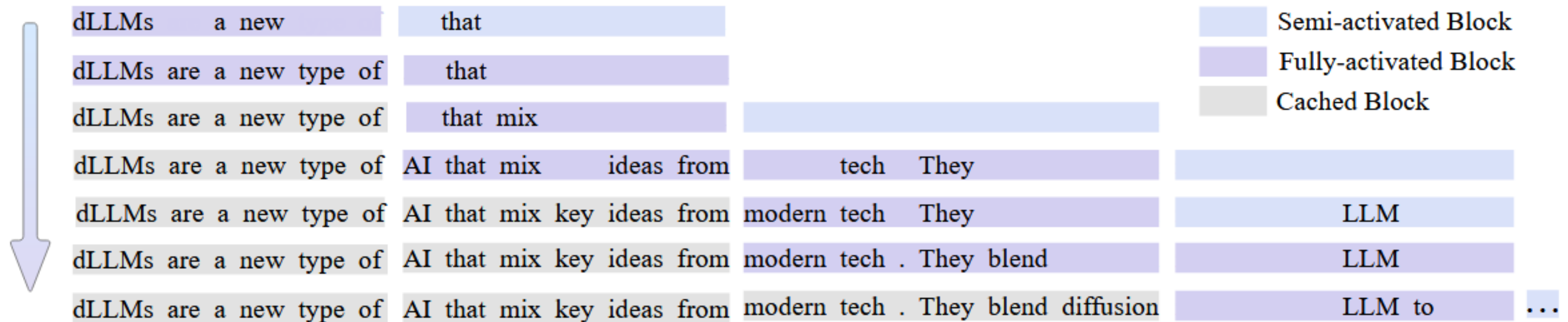
Require: Pre-trained dLLM $p_{\phi-}$; D2F dLLM p_{θ} ; block size k ; training dataset $\mathcal{D}$.

- 1: **while** training **do**
 - 2: Sample a sequence Y from $\mathcal{D}$.
 - 3: Divide Y into N blocks $\{Y_{B_1}, \dots, Y_{B_N}\}$, each of size k .
 - 4: Sample a monotonic noise schedule $\{t_1, \dots, t_N\}$ where $t_1 < \dots < t_N$.
 - 5: For each $i \in \{1, \dots, N\}$, corrupt Y_{B_i} to $Y_{B_i}^{t_i}$ using Eq. 1.
 - 6: Predict distributions for each block $Y_{B_i}^0$ with:
 - 7: Teacher (global view): $p_{\phi-}(Y_{B_i}^0 | Y_{B_1}^{t_1}, \dots, Y_{B_N}^{t_N})$
 - 8: Student (causal view): $p_{\theta}(Y_{B_i}^0 | Y_{B_1}^{t_1}, \dots, Y_{B_i}^{t_i})$
 - 9: Update p_{θ} based on the loss $\mathcal{L}_{\text{D2F}}$ defined in Eq. 3.
-

- Asymmetric distillation 과정을 통해 D2F 모델이 inference단계에서 KV 캐시를 활용하면서도 블록 병렬 디코딩을 할수 있게 됨

Method (3) Pipelined Parallel Decoding

- D2F 추론을 위해 파이프라인 병렬 디코딩 알고리즘을 제안
 - 활성 블록 윈도우(sliding window)를 유지
 - 마지막 블록의 디코딩 진행률이 임계값 τ_{add} 를 넘으면 새로운 블록을 추가
(매번 전체 시퀀스를 다 유지하지 않아도 되어 계산량 감소)



- Dual-State Decoding : 새로운 블록 추가 시 디코딩 수준을 2가지로 나눔
 - Semi-activated: 보수적으로 병렬 디코딩, $\text{confidence} > \tau_{conf}$ 인 토큰만 선택
 - Fully-activated: 이전 블록이 τ_{act} 이상 진행되면 새로운 블록이 디코딩 시작, confidence 높은 토큰이 없더라도 가장 높은 토큰 강제 선택

Method (3) Pipelined Parallel Decoding

- D2F 추론을 위해 파이프라인 병렬 디코딩 알고리즘을 제안

Algorithm 2 Pipelined Parallel Decoding for D2F Inference

Require: D2F model p_θ ; thresholds $\tau_{\text{add}}, \tau_{\text{act}}, \tau_{\text{conf}}$.

- 1: Initialize $Y = \{Y_{B_1}\}$ with a block of mask tokens. 완전히 마스킹된 블록으로 시작
 - 2: **while** generation is not complete **do**
 - 3: **if** the ratio of decoded tokens in $Y_{B_{-1}}$ exceeds τ_{add} and $\langle \text{EOS} \rangle$ not in Y **then** 조건(τ_{add}) 충족 시 새로운 블록 추가
 - 4: Append a new fully masked block with semi-activated state to Y . 새 블록은 semi-activated 상태로 들어옴
 - 5: Forward pass of Y with D2F dLLM p_θ using cached KV
 - 6: **for** the active block Y_{B_i} in Y **do** Semi-activated블록은 confidence > τ_{conf} 일 경우만 토큰 확정
 - 7: Let J_i record the set of token positions in B_i with $> \tau_{\text{conf}}$ prediction confidence
 - 8: **if** Y_{B_i} is fully-activated and J_i is $\emptyset$ **then**
 - 9: Add the token position with the highest confidence to J_i
 - 10: Sample tokens with positions in J_i and remask other positions 그렇지 않은 토큰은 재 마스킹 하여 다음스텝에서 예측
 - 11: **if** the ratio of decoded tokens in $Y_{B_{i-1}}$ exceeds τ_{act} **then**
 - 12: Set Y_{B_i} to be fully-activated 이전블록이 충분히 진행(τ_{act}) 시 Fully-activated 상태로 전환, 공격적 토큰 확정
 - 13: Update KV cache for completed blocks
-

Method (3) Pipelined Parallel Decoding

- 주요 하이퍼파라미터의 성능, 속도 영향도 분석에 대한 ablation study 결과

$\tau_{act} = \tau_{conf}$	τ_{add}	TPS $\uparrow$	Score $\uparrow$
0.95	0.95	105.2	76.9
	0.7	107.2	77.2
	0.5	106.3	77.3
	0.1	104.0	77.7
0.90	0.90	124.5	74.7
	0.7	126.2	75.7
	0.5	124.7	76.2
	0.1	122.1	76.4
0.85	0.85	136.8	72.6
	0.7	139.0	74.2
	0.5	138.5	74.0
	0.1	135.4	75.0

- τ_{conf} : 토큰 확정 confidence threshold
- τ_{act} : 블록이 fully-activated로 전환되는 시점
- τ_{add} : 새로운 블록을 추가하는 기준
- ✓ τ_{act} 를 낮출수록 throughput $\uparrow$, 성능 $\downarrow$
 - 블록을 일찍 fully-activate 시키면 속도 향상, 정확도 하락 trade-off 존재
- ✓ Dual-state ($\tau_{add} < \tau_{act}$) > Single-state ($\tau_{add} = \tau_{act}$)
- ✓ 속도와 품질의 균형 조정 가능

Experiments (1) settings

- 평가 모델: LLaDA-Instruct-8B (Nie et al., 2025), Dream-Base-7B (Ye et al., 2025)
- 훈련 데이터:
 - Bespoke-Stratos-17k (Labs, 2025)에서 파생된 데이터셋 사용
 - HuggingFace Hub에서 공개된 두 가지 컬렉션 활용
 - Qwen2.5-7B가 생성한 응답 포함
 - 사전 필터링: 최대 600 토큰 길이 제한
- LoRA (Low-Rank Adaptation): q_proj, v_proj, k_proj, o_proj 모듈에 적용
 - 랭크(rank): 32, 스케일링: 32, 드롭아웃: 0.1
- 훈련 과정
 - 모든 시퀀스를 512 토큰으로 맞춤 (패딩/트렁케이션)
 - 블록 크기(k): 16
 - 마스크 비율: 0.3 ~ 0.7 (단조 증가)
 - Optimizer: AdamW
 - learning rate = $1e-5$
 - GPU: 8 × NVIDIA A100-40GB
 - 훈련 시간: 12시간
- 추론 하이퍼파라미터 (기본값): $\tau_{conf} = 0.9$, $\tau_{act} = 0.95$, $\tau_{add} = 0.1$

Experiments (2) Main results

- **벤치마크 (Benchmarks)**

- GSM8K (Cobbe et al., 2021)
- GSM8K-CoT (Chain-of-Thought reasoning variant)
- Math (Hendrycks et al., 2021)
- HumanEval (Chen et al., 2021)
- MBPP (Austin et al., 2021b)

- **기준 (Baselines)**

- dLLM: dLLM-Cache (Liu et al., 2025), Fast-dLLM (Wu et al., 2025)
- AR LLM: AR LLMs: LLaMA3-Instruct-8B, Qwen2.5-Base-7B

- **정량적 결과**

- 최초로 오픈소스 dLLM이 AR LLM보다 빠른 추론속도 달성
- 최대 길이를 512로 통일하여 공정 비교

Experiments (2) Main results

- 기존 dLLM vs LLaDA, Dream 모델 기반으로 distillation된 제안 모델의 성능 비교

Test Set	Method	TPS ↑	Latency (s) ↓	Gen. Length	Score ↑
GSM8K 4-shot	LLaDA-Instruct	7.2 (1.0x)	32.3 (1.0x)	231	77.4
	dLLM-Cache	20.1 (2.8x)	11.5 (2.8x)	231	77.5
	Fast-dLLM (Prefix-Cache)	33.3 (4.6x)	7.0 (4.6x)	232	77.8
	Fast-dLLM (Dual-Cache)	35.2 (4.9x)	6.6 (4.9x)	232	78.9
	D2F-LLaDA	52.5 (7.3x)	2.8 (11.5x)	144	77.3
MBPP 3-shot	LLaDA-Instruct	0.9 (1.0x)	71.4 (1.0x)	65	39.0
	dLLM-Cache	2.3 (2.6x)	28.3 (2.5x)	66	37.0
	Fast-dLLM (Prefix-Cache)	13.0 (14.4x)	4.9 (14.6x)	64	37.6
	Fast-dLLM (Dual-Cache)	15.3 (17.0x)	3.8 (18.8x)	58	36.4
	D2F-LLaDA	47.6 (52.9x)	1.4 (51.0x)	68	38.0
HumanEval 0-shot	LLaDA-Instruct	2.8 (1.0x)	38.8 (1.0x)	107	36.0
	dLLM-Cache	4.5 (1.6x)	23.3 (1.7x)	104	39.0
	Fast-dLLM (Prefix-Cache)	13.7 (4.9x)	7.4 (5.2x)	102	38.4
	Fast-dLLM (Dual-Cache)	19.2 (6.9x)	5.2 (7.5x)	100	35.4
	D2F-LLaDA	81.6 (29.1x)	1.6 (24.3x)	133	40.2
Math 4-shot	LLaDA-Instruct	21.1 (1.0x)	11.5 (1.0x)	243	23.7
	dLLM-Cache	26.9 (1.3x)	9.1 (1.3x)	246	23.2
	Fast-dLLM (Prefix-Cache)	47.7 (2.3x)	5.2 (2.2x)	246	22.4
	Fast-dLLM (Dual-Cache)	42.5 (2.0x)	5.8 (2.0x)	246	22.4
	D2F-LLaDA	90.2 (4.3x)	4.3 (2.7x)	384	29.1

D2F-LLADA distillation 모델 성능

Test Set	Method	TPS ↑	Latency (s) ↓	Gen. Length	Score ↑
GSM8K-CoT 8-shot	Dream-Base	9.5 (1.0x)	26.8 (1.0x)	255	75.0
	dLLM-Cache	26.0 (2.7x)	9.8 (2.7x)	255	72.0
	Fast-dLLM (Prefix-Cache)	50.3 (5.3x)	5.1 (5.3x)	255	76.6
	Fast-dLLM (Dual-Cache)	49.8 (5.2x)	5.1 (5.3x)	255	75.0
	D2F-Dream	91.2 (9.6x)	2.8 (9.6x)	256	77.6
MBPP 3-shot	Dream-Base	10.4 (1.0x)	24.6 (1.0x)	256	56.2
	dLLM-Cache	25.5 (2.5x)	10.0 (2.5x)	256	52.6
	Fast-dLLM (Prefix-Cache)	71.6 (6.9x)	3.6 (6.8x)	256	56.4
	Fast-dLLM (Dual-Cache)	73.2 (7.0x)	3.5 (7.0x)	256	51.0
	D2F-Dream	105 (10.1x)	2.3 (10.7x)	240	55.2
HumanEval 0-shot	Dream-Base	20.2 (1.0x)	12.6 (1.0x)	255	54.3
	dLLM-Cache	23.2 (1.1x)	11.0 (1.1x)	255	55.5
	Fast-dLLM (Prefix-Cache)	62.4 (3.1x)	4.1 (3.1x)	255	54.3
	Fast-dLLM (Dual-Cache)	60.0 (3.0x)	4.3 (2.9x)	255	53.0
	D2F-Dream	73.2 (3.6x)	3.1 (4.1x)	227	54.3
Math 4-shot	Dream-Base	9.9 (1.0x)	25.8 (1.0x)	256	35.8
	dLLM-Cache	12.7 (1.3x)	20.2 (1.3x)	256	34.5
	Fast-dLLM (Prefix-Cache)	65.6 (6.6x)	3.9 (6.6x)	256	37.6
	Fast-dLLM (Dual-Cache)	67.0 (6.8x)	3.8 (6.8x)	256	37.1
	D2F-Dream	98.8 (10.0x)	2.6 (9.9x)	256	35.4

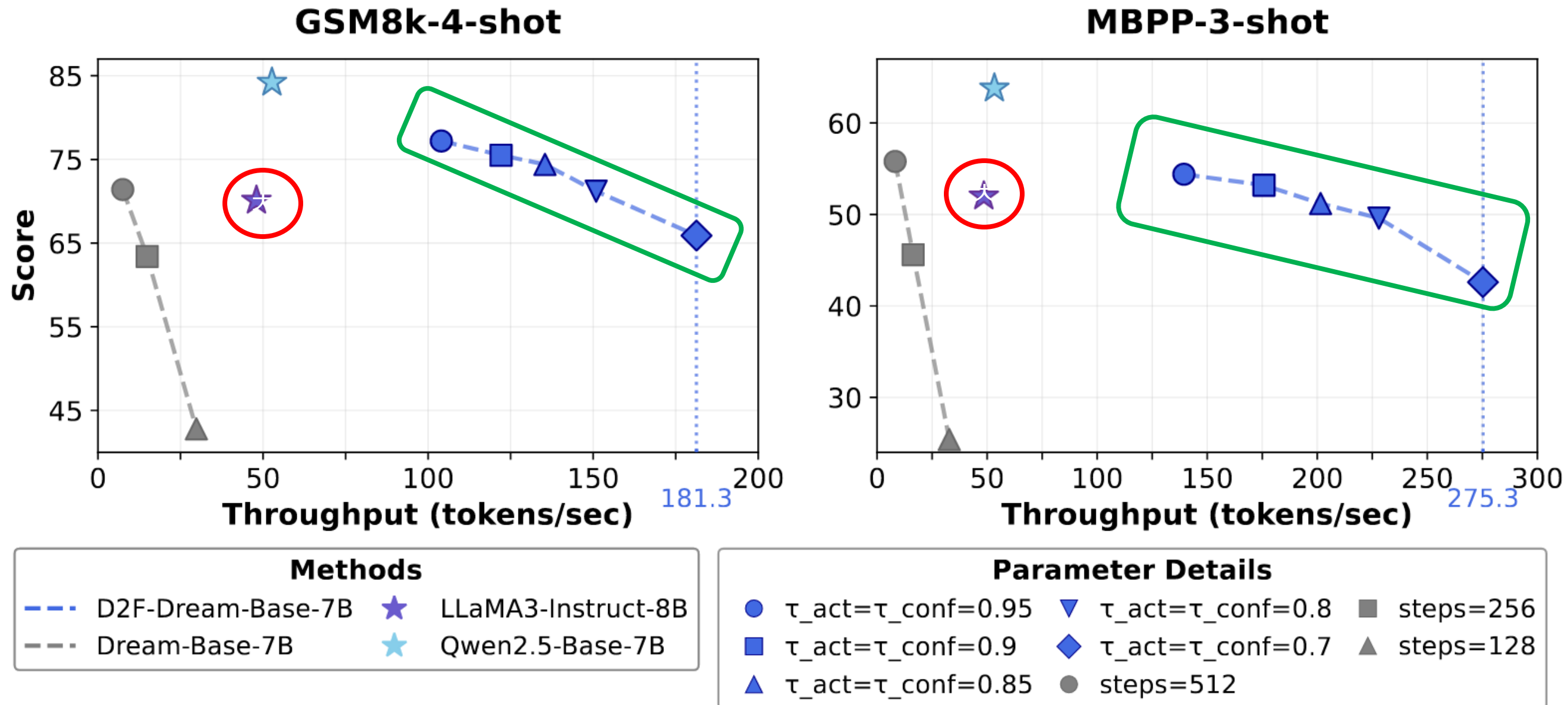
D2F-Dream distillation 모델 성능

➔ 전반적으로 속도가 빨라졌으며, 정확도도 대동소이함

Experiments (3) Ablations & Analysis

- 성능-속도 Tradeoff

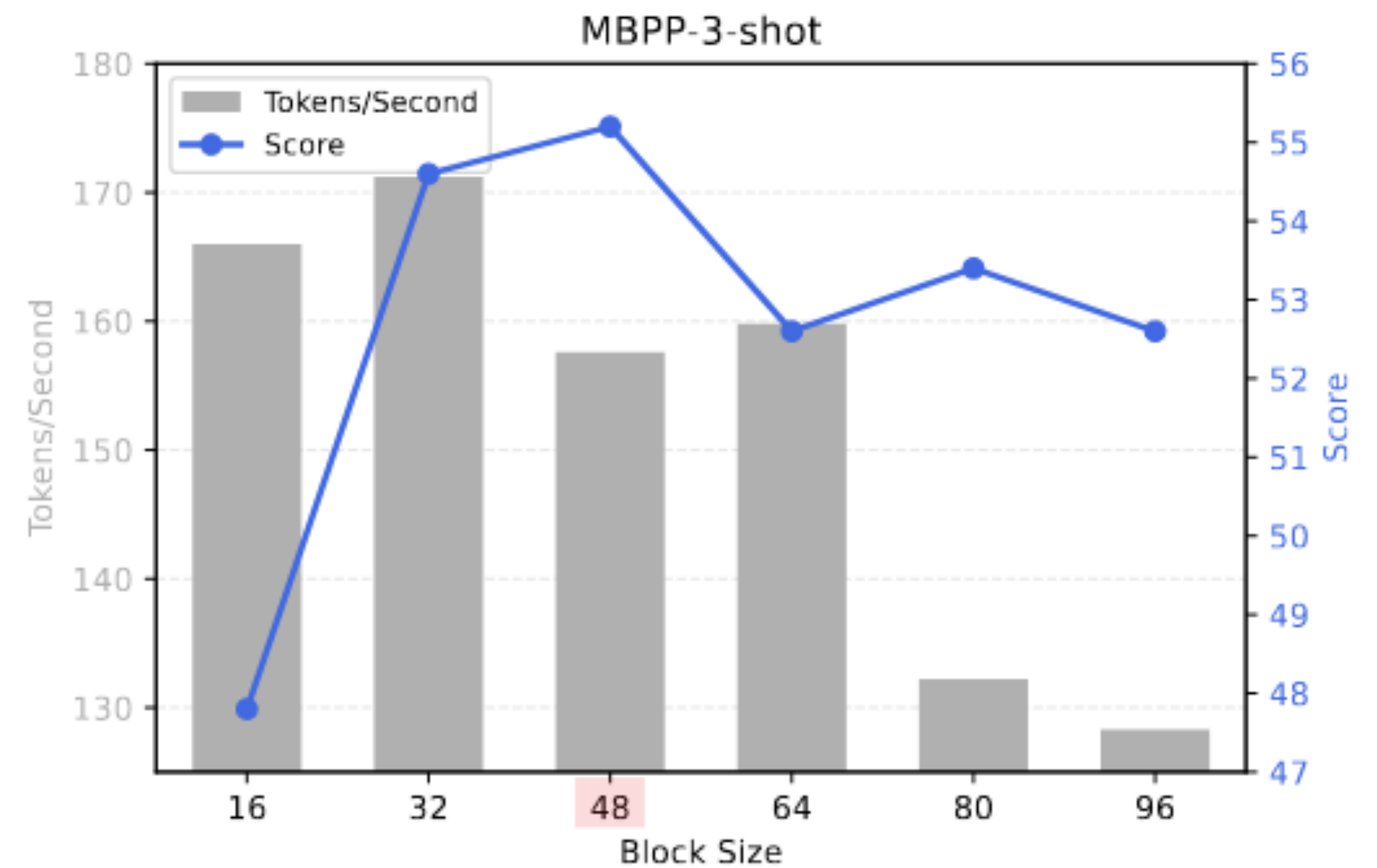
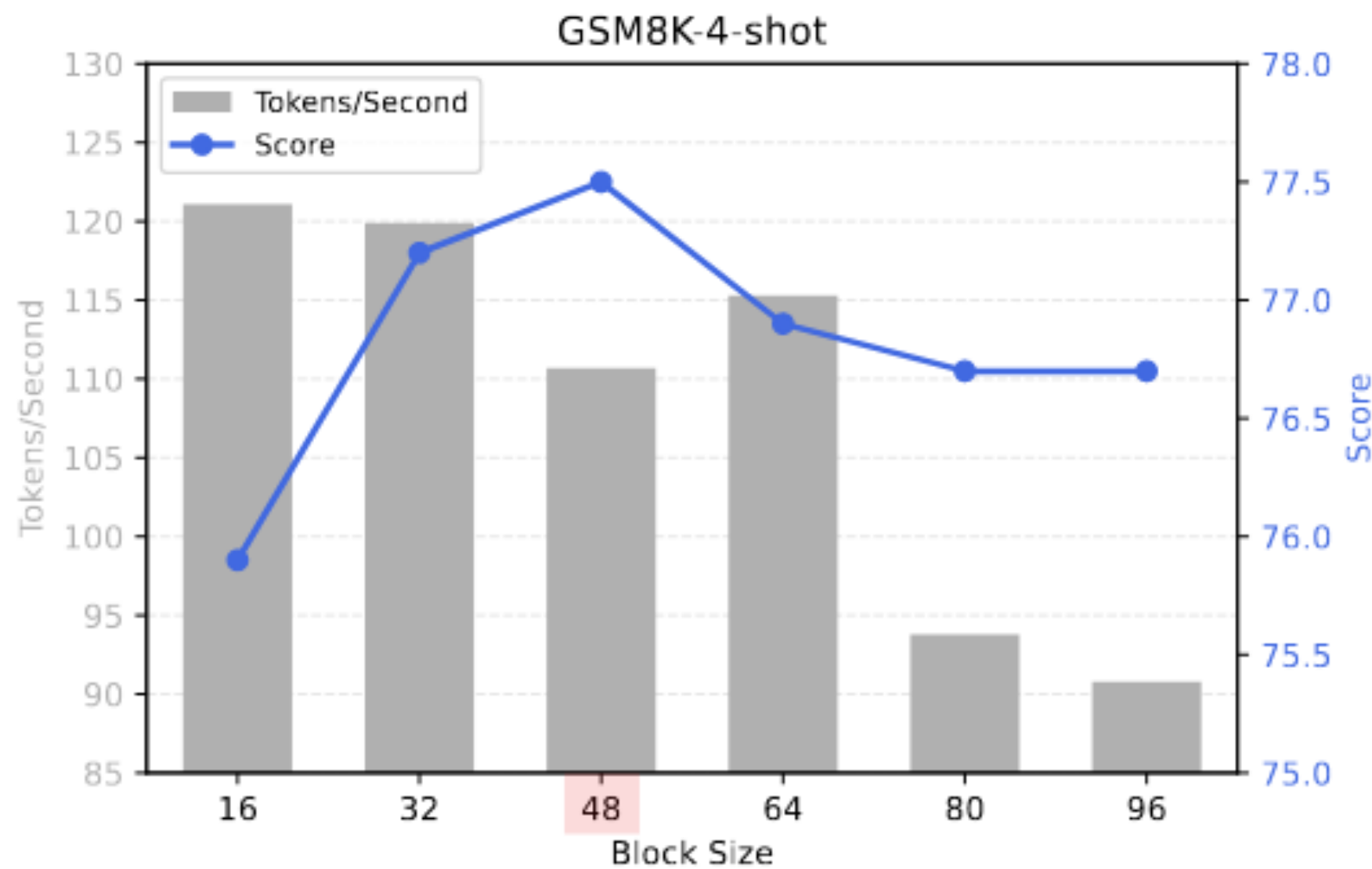
- D2F-Dream-Base-7B, $\tau_{\text{add}}=0.1$, block size = 32
- 결과: LLaMA3-Instruct-8B보다 3.1배 높은 속도에서 더 높은 점수 달성



Experiments (3) Ablations & Analysis

- 블록 크기 영향 (Effect of Block Size)

- 블록 크기 ↑ → 처리량(throughput) ↓
- 적절한 블록 크기에서 성능 최대화 (예: 블록 크기 48 → GSM8K 점수 77.5)



Experiments (3) Ablations & Analysis

- 노이즈 스케줄링 전략

Benchmark	τ_{act}	Model	TPS $\uparrow$	Score $\uparrow$
MBPP 3-shot	0.95	random	147.2	49.6
		D2F	171.2	54.6
	0.90	random	148.5	48.6
		D2F	175.5	53.2
	0.85	random	150.6	47.6
		D2F	177.5	52.6
GSM8k 4-shot	0.95	random	114.5	77.1
		D2F	119.9	77.2
	0.90	random	116.7	76.5
		D2F	123.5	76.4
	0.85	random	118.8	75.9
		D2F	124.8	75.4

- 단조 증가 마스킹(D2F) vs. 랜덤 마스킹 비교
- D2F가 훨씬 우수
 - MBPP에서 점수 +5.0, TPS +24.0 향상

Conclusion

- D2F라는 새로운 훈련 패러다임을 제안
 - 이전 블록에서 부분적으로 예측된 토큰들을 조건으로 다음 블록을 예측하도록 모델을 학습시키는 것
- D2F 적용 효과
 - KV 캐시 사용 가능
 - 여러 블록의 병렬 생성 가능
 - 결과적으로 추론 속도 향상
- 실험적으로 다양한 벤치마크에서 D2F가 AR LLM보다 빠른 추론을 지원함을 입증하였음

- 기존 dLLM의 속도 한계를 극복
- AR-확산 하이브리드 패러다임이라는 새로운 방법 제시
- 오픈소스 생태계에서 실제로 활용 가능한 수준의 추론 효율을 보여준 첫 사례